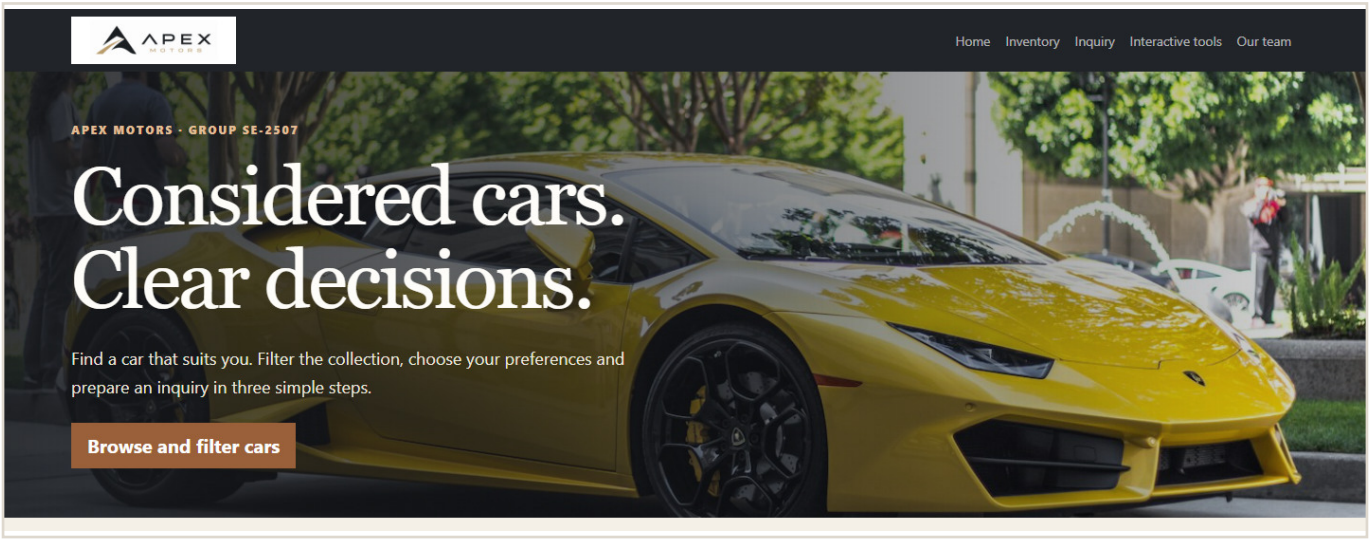
WEB TECHNOLOGIES · ASSIGNMENT 5

JavaScript and DOM Manipulation.

A single-page dealership with DOM updates, event handling, objects, arrays, callbacks, sound and animation.



The home page retains the cream/copper design and full-width vehicle hero.

Team: Apex Motors **Group:** SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad
Prepared: 28 October 2026
Build: HTML, custom CSS and three JavaScript files

Objective. Build a single-page dealership site with all required JavaScript interactions directly available on the main page. Keep the team's familiar colours, vehicle hero, cards and photographs.

Website: https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/
Repository: https://github.com/Vertex-blip/Car-dealership-website



Arsen



Kuttibay



Farabi



Muhammad

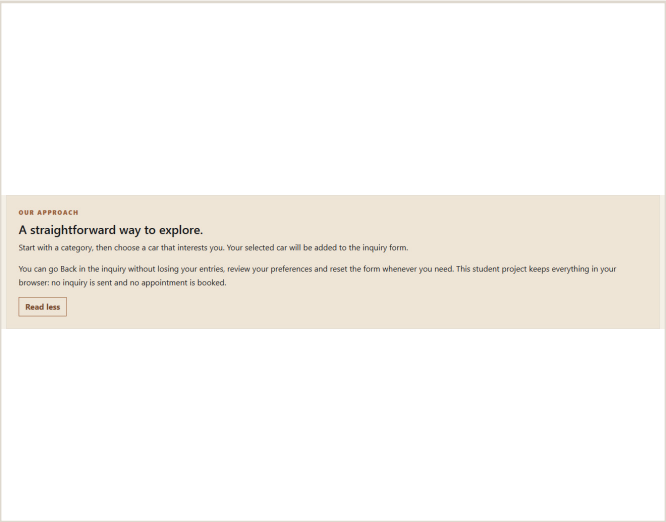
DOM updates and styling

Our approach contains Read More for the content-visibility option. Interactive tools provides a direct background-colour change for the dynamic-styling option.

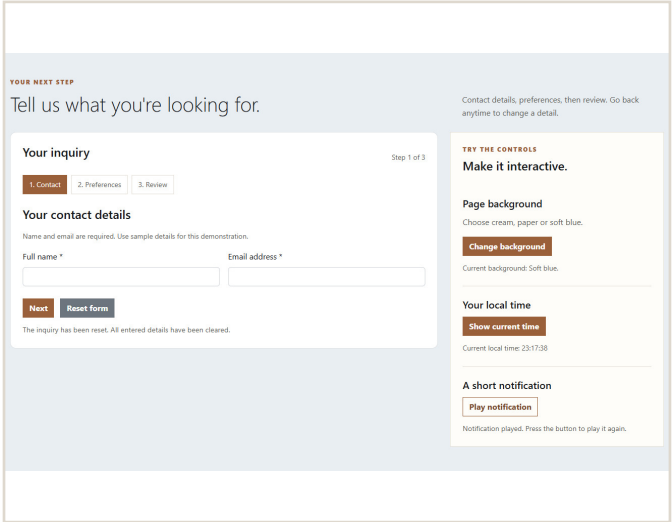
1. Select the Read More button and extra text; attach a click listener.

2. Change style.display and the button state to reveal or hide the extra content.

3. Cycle a small array of colours and assign the selected colour to body.style.backgroundColor.



Our approach: additional content revealed.



Interactive tools: background changed by JavaScript.

js/site.js

```
 92      readMoreButton.setAttribute('aria-expanded', 'false');
 93      readMoreButton.addEventListener('click', () => {
 94        const expanded = readMoreButton.getAttribute('aria-expanded') !== 'true';
 95        readMoreContent.style.display = expanded ? 'block' : 'none';
 96        readMoreButton.setAttribute('aria-expanded', String(expanded));
 97        readMoreButton.textContent = expanded ? 'Read less' : 'Read more';
 98        if (expanded) window.Apex.animate(readMoreContent);
 99      });
100    }
```

js/site.js

```
111      backgroundStatus.textContent = `Current background: ${colors[currentColor].name}.`;
112      backgroundButton.addEventListener('click', () => {
113        currentColor = (currentColor + 1) % colors.length;
114        const color = colors[currentColor];
115        document.body.style.backgroundColor = color.value;
116        backgroundStatus.textContent = `Current background: ${color.name}.`;
117      });
118    }
```

Actual source: js/site.js

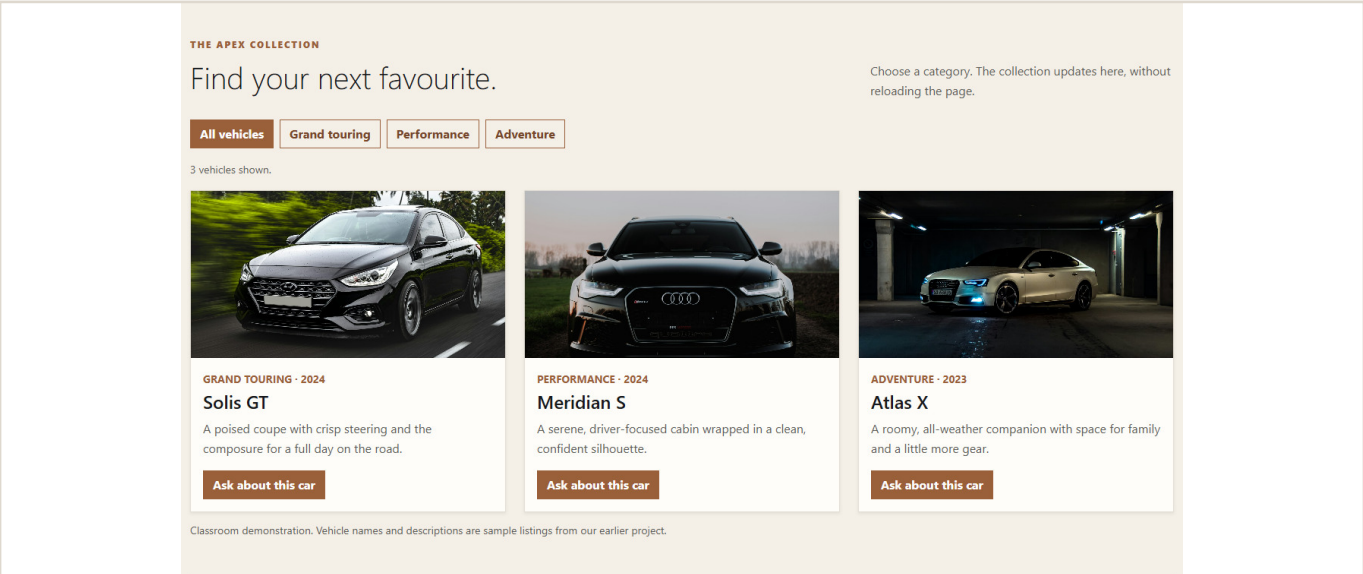
Objects, arrays and vehicle cards

The three inventory cards are generated from structured vehicle data. The familiar square card design and original photographs are retained.

1. Store each vehicle's name, category, photo and description in an object with a label method.

2. Keep the vehicle objects in an array and create DOM cards in an explicit for...of loop.

3. Read object properties and insert content with textContent rather than user-written HTML.



Inventory: the original three cards, now rendered from objects.

js/site.js

```
  6      return {
  7        id, name, category, categoryLabel, year, image, alt, description,
  8        label() {
  9          return `${this.categoryLabel} · ${this.year}`;
 10        }
 11      };
```

js/site.js

```
 15    const vehicles = [
 16      createVehicle('solis', 'Solis GT', 'touring', 'Grand touring', 2024,
 17        'images/car-01.jpg', 'Black sedan driving along a tree-lined road',
 18        'A poised coupe with crisp steering and the composure for a full day on the road.'),
 19      createVehicle('meridian', 'Meridian S', 'performance', 'Performance', 2024,
```

js/inventory.js

```
 51      const selectedVehicles = vehicles.filter(matchesCategory);
 52      const fragment = document.createDocumentFragment();
 53      for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
 54      grid.replaceChildren(fragment);
 55      count.textContent = `${selectedVehicles.length} ${selectedVehicles.length === 1 ?
     'vehicle' : 'vehicles'} shown.`;
 56      if (animate) window.Apex.animate(grid);
```

Actual source: js/site.js, js/inventory.js

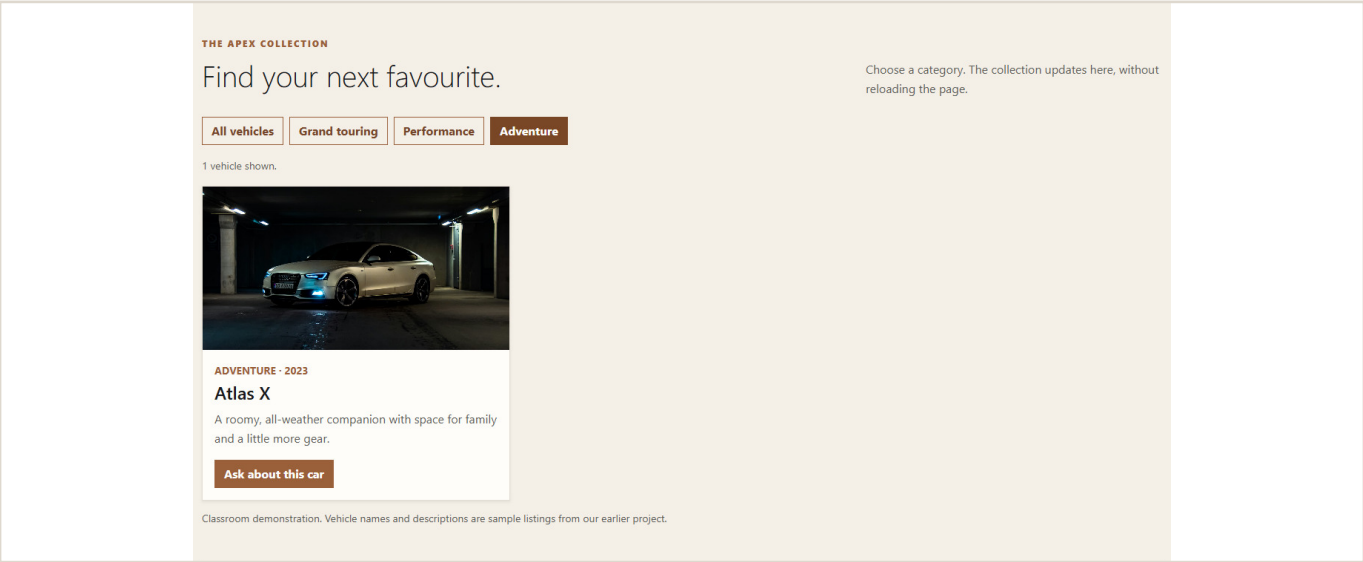
Category filtering

Category buttons filter the existing collection without a page reload. The result count and visible cards respond to the selected category.

1. Listen for category button clicks and read the selected category.

2. Use a switch statement to choose the matching category condition.

3. Pass that condition to filter, then render the resulting array and update the result count.



Inventory: a selected category updates the collection.

js/inventory.js

```
19    function matchesCategory(vehicle) {
20      // Each switch branch controls which objects reach the visible card grid.
21      switch (category) {
22        case 'touring': return vehicle.category === 'touring';
23        case 'performance': return vehicle.category === 'performance';
24        case 'adventure': return vehicle.category === 'adventure';
25        default: return true;
26      }
27    }
28
29    function makeCard(vehicle) {
30      const card = element('article', 'vehicle-card card');
```

js/inventory.js

```
49    function render(animate = true) {
50      // filter is higher-order: it receives matchesCategory as its predicate callback.
51      const selectedVehicles = vehicles.filter(matchesCategory);
52      const fragment = document.createDocumentFragment();
53      for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
54      grid.replaceChildren(fragment);
55      count.textContent = `${selectedVehicles.length} ${selectedVehicles.length === 1 ?
    'vehicle' : 'vehicles'} shown.`;
56      if (animate) window.Apex.animate(grid);
```

Actual source: js/inventory.js

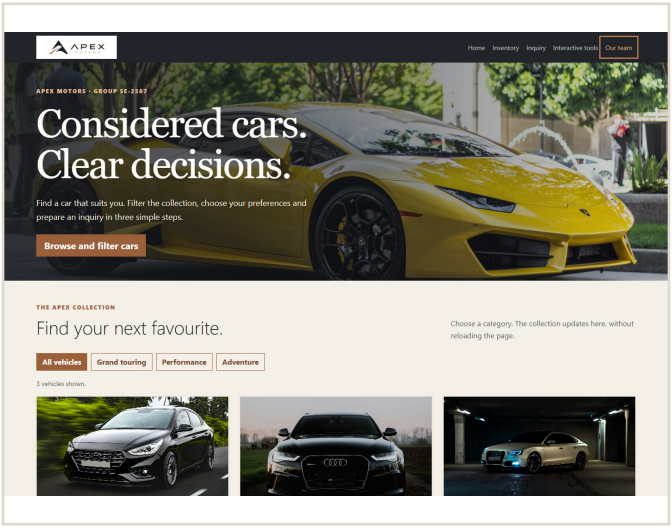
Button and keyboard events

The five navigation links lead to sections on this page and support the required arrow-key focus movement. Buttons also update the page through event listeners.

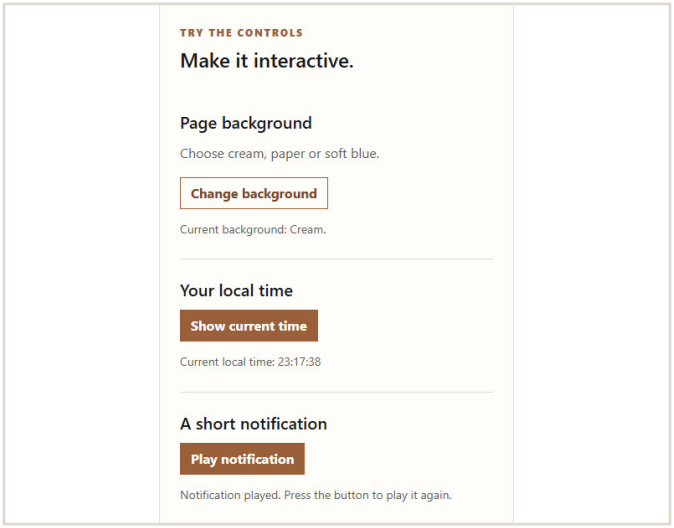
1. Select the navigation links and register a keydown listener on the menu.

2. Detect arrow keys, calculate the next or previous index with wraparound, and focus that link.

3. Use the time button's click callback to show a newly formatted current time.



A menu link receives keyboard focus.



Interactive tools: working time and sound actions.

```
js/site.js
 56      navigation.addEventListener('keydown', event => {
 57        const current = links.indexOf(document.activeElement);
 58        if (current === -1) return;
 59        let next;
 60        switch (event.key) {
 61          case 'ArrowRight':
 62          case 'ArrowDown': next = (current + 1) % links.length; break;
 63          case 'ArrowLeft':
 64          case 'ArrowUp': next = (current - 1 + links.length) % links.length; break;
 65          case 'Home': next = 0; break;
 66          case 'End': next = links.length - 1; break;
js/site.js
 77        event.preventDefault();
 78        links[next].focus();
 79      });
js/site.js
122    if (timeButton && timeOutput) {
123      timeButton.addEventListener('click', () => {
124        timeOutput.textContent = `Current local time: ${new Date().toLocaleTimeString('en-
     GB')}`;
125        window.Apex.animate(timeOutput);
126      });
```

Actual source: js/site.js

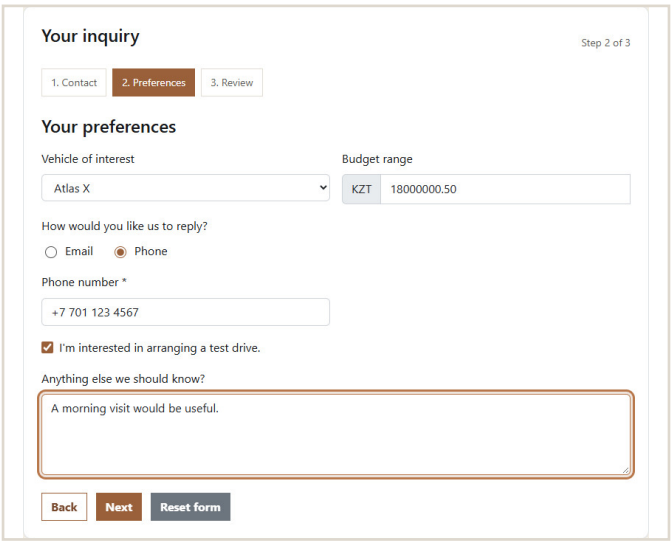
One inquiry, three steps

The multi-step form is the chosen callback option. The original inquiry fields are grouped into contact details, preferences, and review.

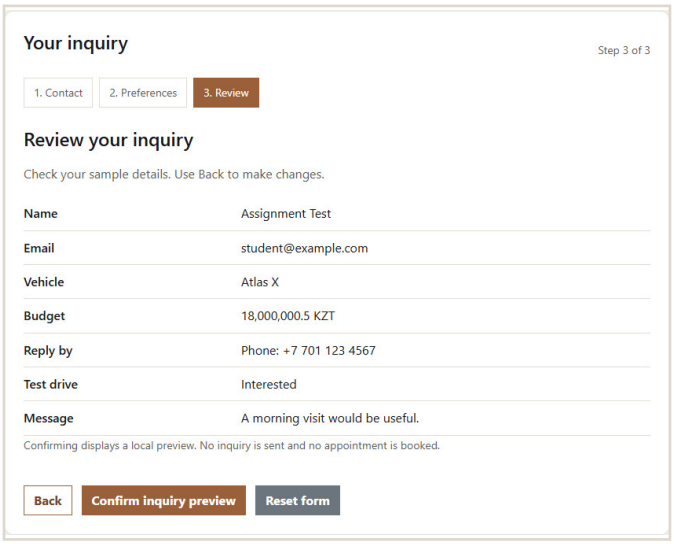
1. Keep the current step in state and validate it before moving forward.

2. Pass the step-display function as a callback to the navigation function.

3. Hide or reveal fieldsets without recreating their inputs, so Back retains entered values.



Step 2: inquiry preferences.



Inquiry: review before confirmation.

js/booking.js

```
133    function navigateStep(direction, onStepChange) {
134      if (direction > 0 && !validateStep(currentStep)) {
135        status.textContent = 'Please correct the highlighted fields before continuing.';
136        return;
137      }
138      const nextStep = Math.max(0, Math.min(2, currentStep + direction));
139      onStepChange(nextStep);
140    }
141    nextButton.addEventListener('click', () => navigateStep(1, showStep));
142    previousButton.addEventListener('click', () => navigateStep(-1, showStep));
143
```

js/booking.js

```
106    function showStep(index, moveFocus = true) {
107      currentStep = index;
108      steps.forEach((step, position) => { step.hidden = position !== currentStep; });
109      previousButton.hidden = currentStep === 0;
110      nextButton.hidden = currentStep === 2;
111      submitButton.hidden = currentStep !== 2;
112      document.getElementById('inquiryStepCount').textContent = `Step ${currentStep + 1} of
     3`;
113      document.querySelectorAll('[data-step-label]').forEach((label, position) => {
114        if (position === currentStep) label.setAttribute('aria-current', 'step');
115        else label.removeAttribute('aria-current');
```

Actual source: js/booking.js

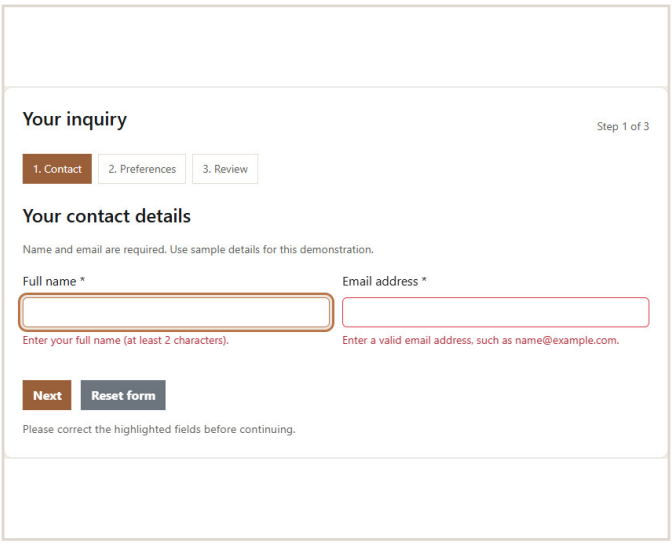
Validation, review and reset

The form checks required contact values and the user's preferences, then displays a local preview result. It does not send personal information to a server.

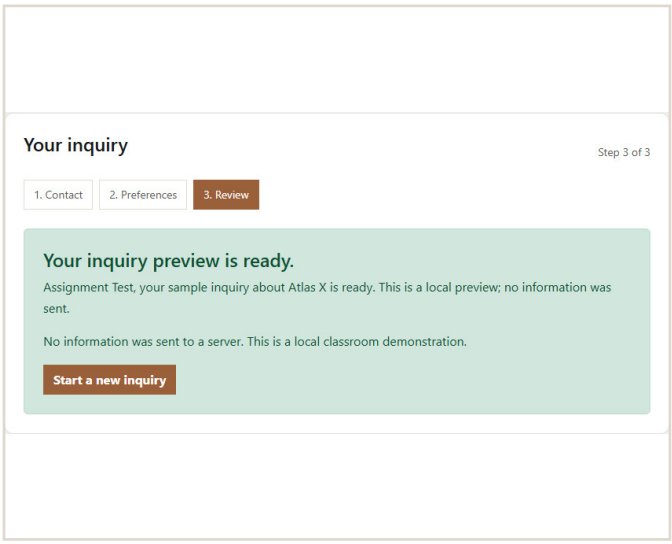
1. Prevent default submission, display field errors, and focus the first invalid control.

2. Build the review from the entered values and show confirmation without reloading.

3. Reset all fields, error messages, and step state when the user starts again.



Missing or invalid values receive clear feedback.



A completed inquiry preview remains on the page.

js/booking.js

```
 66    function validateStep(index, focusFirst = true) {
 67      let firstInvalid = null;
 68      for (const id of fieldsByStep[index]) {
 69        if (!setError(id, fieldError(id)) && !firstInvalid) firstInvalid =
    document.getElementById(id);
 70      }
 71      if (focusFirst && firstInvalid) firstInvalid.focus();
 72      return !firstInvalid;
 73    }
 74
 75    function clearResolvedStatus() {
 76      // Read validity without adding new error messages before a navigation attempt.
```

js/booking.js

```
144    function clearValues() {
145      form.reset();
146      form.querySelectorAll('input, textarea').forEach(input => {
147        if (input.type === 'radio') input.checked = input.id === 'replyEmail';
148        else if (input.type === 'checkbox') input.checked = false;
149        else input.value = '';
150      });
151      vehicleSelect.value = '';
152      fieldsByStep.flat().forEach(id => setError(id, ''));
153      document.getElementById('inquirySummary').replaceChildren();
```

Actual source: js/booking.js

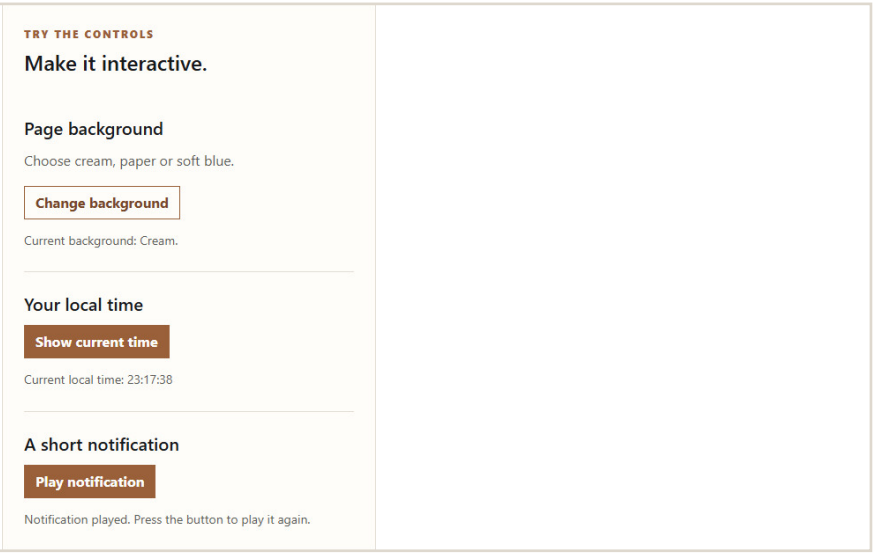
Sound and event-triggered animation

Sound and motion are attached to intentional user actions. The sound file is included locally and playback is reused instead of creating overlapping clips.

1. Load one Audio player and start it only after the notification button is clicked.

2. Pause and rewind the player before each playback; report playback failures clearly.

3. Animate content, inventory or form changes and respect the reduced-motion preference.



Interactive tools: user-triggered notification sound and feedback.

js/site.js

```
133      const sound = new Audio('assets/chime.wav');
134      sound.preload = 'auto';
135      sound.volume = 0.4;
136      let requestNumber = 0;
137      async function playChime() {
138        const currentRequest = ++requestNumber;
139        sound.pause();
140        sound.currentTime = 0;
141        try {
142          await sound.play();
143          if (currentRequest !== requestNumber) return;
144          soundStatus.textContent = 'Playing the notification sound.';
```

js/site.js

```
 30      animate(element) {
 31        if (!element || !element.animate
 32            || window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;
 33        animations.get(element)?.cancel();
 34        const animation = element.animate(
 35          [{opacity: 0, transform: 'translateY(10px)'}, {opacity: 1, transform:
    'translateY(0)'}],
 36          {duration: 260, easing: 'ease-out'}
 37        );
 38        animations.set(element, animation);
```

Actual source: js/site.js

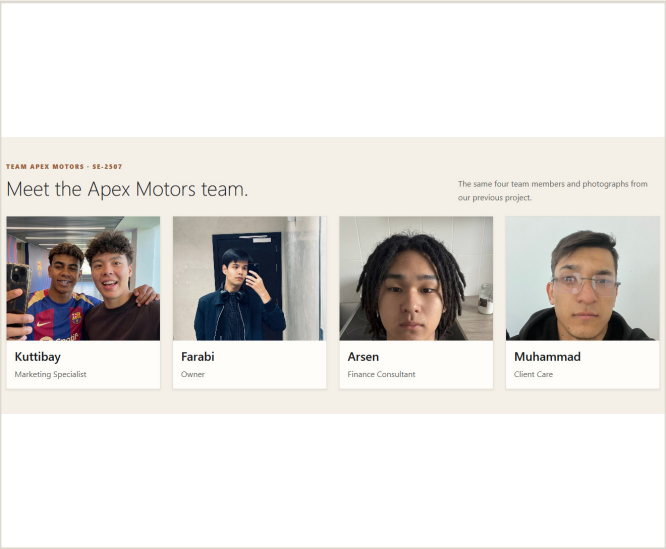
One page, separate source files

HTML, CSS and JavaScript are kept separate. All required features share one main page. The colour palette, hero, square cards and photographs continue the team's earlier design.

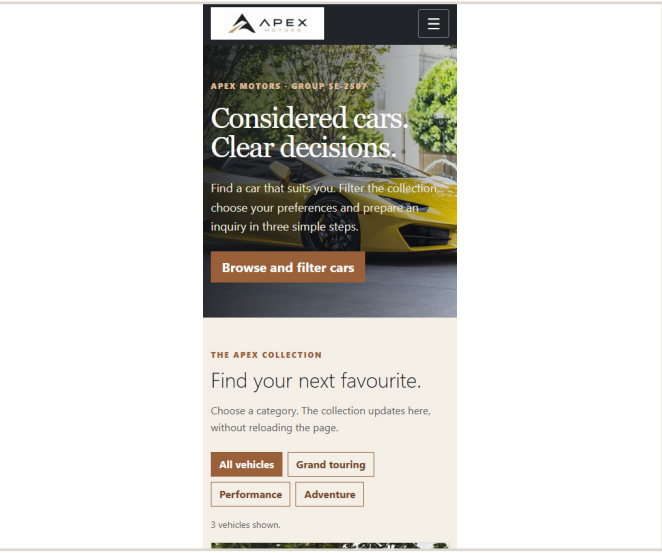
1. Link the custom stylesheet and three external deferred scripts from index.html.

2. Use section anchors, visible labels, alternative text, keyboard focus and a custom mobile menu.

3. Check every interaction and the four original team portraits at desktop and phone widths.



The same four team members and portraits.



Responsive main page at phone width.

index.html

```
12      <link rel="stylesheet" href="styles.css?v=plain-a5" />
13      <script src="js/site.js?v=plain-a5" defer></script>
14      <script src="js/inventory.js?v=plain-a5" defer></script>
15      <script src="js/booking.js?v=plain-a5" defer></script>
16    </head>
17    <body>
18      <a class="skip-link" href="#main-content">Skip to main content</a>
19      <header class="site-header">
```

styles.css

```
 2  :root {
 3    --apex-ink: #17191c;
 4    --apex-copper: #9a603a;
 5    --apex-cream: #f5f0e7;
 6    --apex-paper: #fffdf9;
 7    --apex-muted: #5c5c59;
 8    --apex-border: #e3ddd3;
 9  }
10
```

Actual source: index.html, styles.css

Verification and reflection

Recorded verification: 231/231 checks passed across 3 browser runs; 0 runtime/resource errors.

Evidence. Screenshots show the resulting pages; source screenshots use literal current code with file names and line numbers. Browser results are stored in screenshots/test-results.json.

Live verification. 77/77 checks passed; 0 runtime/resource errors.

Deployment. Application commit 4646afc was pushed to the shared main branch. GitHub Pages workflow 40744013953 completed successfully. The standalone HTML/CSS/JavaScript page returned HTTP 200. All 77 live browser checks passed, including filtering, keyboard navigation, multi-step inquiry, audio, animations and narrow-screen layouts. No framework resources, external dependencies, JavaScript errors or missing resources were recorded.

DOM and styling. Read More visibility, dynamic inventory text, result counts and background colour.

Events and data. Click listeners, arrow-key focus, object methods, arrays, loops, filter and switch.

Inquiry. Invalid values, Next and Back, retained values, review, confirmation and reset.

Sound, motion and layout. Actual audio playback, repeated-click restart, transitions, offline assets and responsive sections.

Challenges and solutions. Keeping the familiar visual design while making all features visible on one page requires clear sections and reusable CSS rules. The scripts target each interactive component. Form steps can hide invalid controls, so validation identifies and focuses the correct field before advancing. Repeated sound clicks are handled by restarting one player.

Reflection. Reusable functions and structured vehicle data make the added features easier to understand and change. A single main page also makes the required interactions easy to find. The inquiry remains an honest local demonstration.

Defense and submission. Every member should explain the code and practise a small change. The defense is worth 40%. Each student submits the PDF and ZIP individually and attends the practice lesson. The supplied LMS screenshot shows 29 October 2026, 14:00; check the LMS for the current deadline.